Flock Energy — Engineering Take-Home

Urja Meter Ops Modern API Bridge & Operational Dashboard

Candidate: Jayendra Singh

GitHub Repository: https://github.com/Jayendra-singh-k98/flock-energy-takehome

Target Portal: `https://urja-ops.flockenergy.tech`

Stack: Python 3.12, FastAPI, Pydantic v2, Requests, Pytest, Vanilla CSS, Leaflet.js, Chart.js

Status: 21 Tests Passed (100%) All 6 Extensions Built

1. Executive Summary

The internal web portal **"Urja Meter Ops"** is used by utility staff to look up smart meters, network locations, and energy consumption. Because it lacked programmatic API access, teams previously copy-pasted data from browsers.

This project provides a clean, documented, high-performance API service built in front of Urja Meter Ops. The solution reverse-engineers the portal's internal endpoints, bypasses SvelteKit CSRF restrictions, signs bulk export requests using the portal's dynamic HMAC-SHA256 protocol, normalizes legacy and v2 data models, reconstructs the complete 7-tier distribution hierarchy, and serves an interactive web dashboard with geospatial mapping and telemetry visualization.

2. Portal Reverse-Engineering & Protocol Discovery (PROTOCOL.md)

A. Architecture & Session Lifecycle

- **Portal Framework:** The portal runs on SvelteKit with client-side SPA routing and SSR hydration.
- **CSRF Origin Protection:** Initial programmatic `POST /login` calls failed with HTTP `403: Cross-site POST form submissions are forbidden`. SvelteKit requires an `Origin` header matching the portal host. Adding `Origin: https://urja-ops.flockenergy.tech` resolved the issue.
- **Session Cookie:** On successful login, the server sets `__Secure-better-auth.session_token` (TTL 3,600s / 1 hour). The `PortalClient` tracks session age and automatically re-authenticates on 401/403 or login redirects.

B. Cryptographic Bulk Export Signing Protocol

Static analysis of frontend chunk `nodes/4.2Bgc2kUI.js` revealed an export RPC mechanism requiring an HMAC-SHA256 signature:

1. The client fetches an active signing key via `GET /portal/keys`.
2. It constructs a signature payload: `"{METHOD}\n{PATH}\n{QUERY}\n{TIMESTAMP}"` (e.g. `"GET\n/portal/export\npage=1\n1761648000"`).

3. It calculates an HMAC-SHA256 digest using the UTF-8 secret key.
4. It sends the headers `x-timestamp` and `x-signature` to `GET /portal/export?page=1`.

Impact: Ingests all **403 meters** in a single call, avoiding 403 individual page scrapes.

C. Legacy vs. Modern "v2" Data Normalization

- **Legacy Meters (238 units):** Store nameplate data in a parameter list (`{"parameterName": "...", "parameterValue": "..."}`).
- **Modern v2 Meters (165 units):** Store nameplate data inside an escaped JSON string under `classData` .
- Our service normalizes both formats into unified, strongly typed Pydantic models.

3. Discovered Endpoints Catalog

Endpoint	Method	Auth	Description
`/login`	POST	Origin header	Authenticates operator credentials and sets session cookie.
`/portal/meters/search`	GET	Cookie	Search meters by query substring (`q`) and page (`page`).
`/portal/dts`	GET	Cookie	Paginated Distribution Transformers (40 DTs total across 2 pages).
`/portal/keys`	GET	Cookie	Retrieves the active HMAC-SHA256 signing secret.
`/portal/export`	GET	Cookie + HMAC	Signed bulk export returning all 403 meters with hierarchy and geo coordinates.
`/portal/meters/{id}/geo`	GET	Cookie	Returns meter latitude and longitude coordinates.
`/portal/meters/{id}/energy`	GET	Cookie	Returns 337 interval telemetry readings (7 days of 30-min readings: kWh, kVAh, voltR).

4. Features & Implemented Extensions

1. Core REST API

- `GET /api/v1/meters` : Multi-attribute filtering (make, phase type, status, DT, feeder, zone, search query) with pagination.
- `GET /api/v1/meters/{id}` : Full meter specification, hierarchy tree path, and latest telemetry reading.
- `GET /api/v1/transformers` : All 40 DTs with aggregate meter counts (installed, faulty, decommissioned).
- `GET /api/v1/transformers/{code}` : DT details with complete list of connected meters.

2. Network Hierarchy Reconstruction (Extension 1)

Reconstructs the full 7-tier distribution network and calculates aggregate downstream meter counts:

```
Zone (3) ➔ Circle (6) ➔ Division (10) ➔ Subdivision (14) ➔ Substation (18) ➔ Feeder (28) ➔ DT
```

Exposed via `GET /api/v1/hierarchy` and subtree queries via `GET /api/v1/hierarchy/{level}/{code}` .

3. Spatial Proximity Search (Extension 2)

Answers "*What meters are near this location?*" using the spherical Haversine distance formula:
`GET /api/v1/spatial/nearby?latitude=26.9389&longitude=75.8309&radius_meters=3000&limit=10`

4. Telemetry Analytics & Voltage Anomaly Detection (Extension 3)

Analyzes 337 interval telemetry readings per meter, calculating total active kWh delta, average/minimum/maximum Phase R voltage, power factor ratios, and flagging voltage anomalies outside nominal tolerance (210V - 250V).
Exposed via `GET /api/v1/meters/{id}/energy`.

5. Freshness, In-Memory Caching & Long-Running Robustness (Extension 4 & 5)

- Configurable TTL cache (default 15 minutes) with automatic staleness detection.
- On-demand invalidation via `POST /api/v1/sync` and health metrics via `GET /api/v1/sync/status`.
- Mutex-locked session renewal recovering from token expiry without race conditions.

6. Modern Web Client Dashboard (Extension 6)

Served directly at `http://localhost:8000/`:

- **Fleet KPI Cards:** Live counts of all 403 meters (238 Installed, 90 Faulty, 75 Decommissioned) and 40 DTs.
- **Interactive Leaflet Map:** Color-coded meter markers across Jaipur city with popups.
- **Chart.js Telemetry Graph:** Dual-axis time-series visualization for active kWh and Phase R voltage.
- **Collapsible Hierarchy Tree:** Interactive navigation through all 7 network tiers.

5. Test Suite & Verification

A comprehensive automated test suite of **21 unit and integration tests** verifies all API contracts, authentication flows, HMAC signing, spatial mathematics, and edge cases:

```
============================= test session starts ==============================
tests/test_analytics_api.py::test_fleet_overview PASSED                  [  4%]
tests/test_analytics_api.py::test_fleet_anomalies PASSED                 [  9%]
tests/test_analytics_api.py::test_health_check PASSED                    [ 14%]
tests/test_analytics_api.py::test_sync_status PASSED                     [ 19%]
tests/test_client.py::test_hmac_signature_generation PASSED              [ 23%]
tests/test_client.py::test_client_headers PASSED                         [ 28%]
tests/test_hierarchy_api.py::test_get_full_hierarchy PASSED              [ 33%]
tests/test_hierarchy_api.py::test_get_hierarchy_node PASSED              [ 38%]
tests/test_hierarchy_api.py::test_get_hierarchy_node_not_found PASSED    [ 42%]
tests/test_meters_api.py::test_list_meters_default PASSED                [ 47%]
tests/test_meters_api.py::test_list_meters_filtering PASSED              [ 52%]
tests/test_meters_api.py::test_list_meters_search PASSED                 [ 57%]
tests/test_meters_api.py::test_get_single_meter PASSED                   [ 61%]
tests/test_meters_api.py::test_get_meter_not_found PASSED                [ 66%]
tests/test_meters_api.py::test_get_meter_geo PASSED                      [ 71%]
tests/test_meters_api.py::test_get_meter_energy PASSED                   [ 76%]
tests/test_spatial_api.py::test_haversine_distance_calculation PASSED    [ 80%]
tests/test_spatial_api.py::test_nearby_meters_query PASSED               [ 85%]
tests/test_transformers_api.py::test_list_transformers PASSED            [ 90%]
tests/test_transformers_api.py::test_get_transformer_detail PASSED       [ 95%]
tests/test_transformers_api.py::test_get_transformer_not_found PASSED    [100%]

============================== 21 passed in 2.76s ==============================
```

6. Reflection & Self-Critique (REFLECTION.md)

1. What assumptions did you make?

We assumed the legacy portal is strictly read-only and that bulk export (`/portal/export`) serves as the most reliable single source of truth for master meter data. We assumed a 15-minute TTL cache provides an optimal balance between freshness and upstream load protection.

2. Which part was the most difficult, and how did you get unstuck?

Bypassing the SvelteKit CSRF check (which required the `Origin` header) and reverse-engineering the dynamic HMAC-SHA256 signature formula in `nodes/4.2Bgc2kUI.js`. Decompiling the client JavaScript bundles unlocked the cryptographic signing routine.

3. If you had another day, what would you improve?

Transition the in-memory store to **PostgreSQL with PostGIS** (for R-Tree spatial indexing) and **TimescaleDB** (for time-series interval rollups), add webhook dispatchers for status change notifications, and implement API key authentication.

4. What mistake did you make while solving this?

Encountered a Windows-1252 terminal encoding error (`UnicodeEncodeError: 'charmap'`) when printing raw HTML responses. We resolved it by writing directly to UTF-8 output files and enforcing UTF-8 streams across all modules.

5. Scaling Critique: At what data scale would the current approach struggle?

The current in-memory inverted index handles 403 meters in sub-millisecond query time with < 15MB RAM. However:

- **At 50,000 meters:** In-memory dictionary remains feasible, but spatial calculations (evaluating 50,000 Haversine pairs per request) increase CPU load to ~50ms.
- **At 1,000,000+ meters:** Heap memory exceeds 2GB-4GB, startup pre-warming times out, and linear scans become unviable. The solution at scale is migrating to PostgreSQL 16 with PostGIS GIST spatial indices (`ST_DWithin` reducing spatial complexity from $O(N)$ to $O(\log N)$) and Redis sliding-window caching.

7. Deliverables Index

Deliverable	File Path	Description
Source Code	`app/`	FastAPI service, models, client, services, and web dashboard.
README	`README.md`	Front-door user guide, quickstart, sample curl requests, architecture.
PROTOCOL.md	`PROTOCOL.md`	Technical report detailing portal reverse-engineering and HMAC signatures.
openapi.json	`openapi.json`	Standard OpenAPI 3.1.0 specification describing all 14 endpoints.
REFLECTION.md	`REFLECTION.md`	Answers to all reflection questions and architectural scaling analysis.
Test Suite	`tests/`	21 passing integration and contract tests.